Instituto Tecnológico y de Estudios Superiores de Monterrey

TC2037 - Implementación de Métodos Computacionales

Ian Julián Estrada Castro – A01352823

Actividad Integradora 3 – Programación Paralela

12 de junio de 2024

Tabla de contenido

Concurrencia *3*
Paralelismo *3*
Programación paralela *3*
Programación paralela basada en hilos (threads) **3**
Hilos (Threads) **3**
Exclusión mutua **4**
Deadlocks **4**
Tipos de paralelismos *4*
Paralelismo de datos **4**
Paralelismo de tareas **4**
CUDA (Compute Unified Device Arquitecture) *4*
GPU (Unidad de Procesamiento Gráfico) **4**
Speedup y eficiencia *5*
Actividad integradora (Suma de números primos) *5*
Implementación secuencial **6**
Implementación paralela **6**
Implementación en CUDA **7**
Reflexiones *9*

Concurrencia

La concurrecia se refiere a la capacidad de distintas partes de un programa, algoritmo o problema de ser ejecutado en desorden o en orden parcial, sin afectar el resultado final. Un programa es concurrente si puede soportar dos o más acciones en un curso al mismo tiempo. La concurrencia consiste en tratar muchas cosas a la vez.

Paralelismo

El paralelismo puede confundirse con la concurrencia, pero en este caso, se refiere a la aplicación de múltiples CPU a un problema único, es decir, la ejecución simultánea de varios procesos computacionales. Est significa que varios medios de ejecución física: Varios procesadores, varias computadoras, y la suficiente memoria para guardarlos.
Un programa es paralelo cuando puede soportar dos o más acciones ejecutándose simultáneamente. El paralelismo consiste en hacer muchas cosas a la vez. Por ende, es posible tener concurrencia sin paralelismo

Programación paralela

La computación paralela consiste en el uso simultáneo de varios procesadores o núcleos que ejecutan una serie de instrucciones que conforman las distintas partes en las que se ha descompuesto un problema computacional. (Unir, 2024)

Programación paralela basada en hilos (threads)

La programación basada en hilos utiliza múltiples hilos de ejecución dentro de un solo proceso. Cada hilo puede ejecutar una tarea diferente simultáneamente, compartiendo el mismo espacio de direcciones de memoria del proceso principal. Aquí hay memoria compartida la cual tiene que ser controlada por **exclusión mutua** y cuidando problemas como **interbloqueos (deadlocks).**

Hilos (Threads)

Un hilo (o "thread" en inglés) es la unidad más pequeña y básica de utilización del CPU, que puede ser gestionada de manera independiente por un sistema operativo. Un hilo es parte de un proceso y comparte los recursos de éste, como la memoria y las variables globales, pero puede ejecutar instrucciones de manera concurrente con otros hilos dentro del mismo proceso. La mayoría de las aplicaciones de software que se ejecutan en las computadoras modernas son multihilos. Por lo general, una aplicación se implementa como un proceso separado por varios hilos de control

Exclusión mutua

La exclusión mutua es una técnica utilizada para evitar que múltiples hilos accedan simultáneamente a una sección crítica del código o a recursos compartidos. El objetivo es asegurarse de que solo un hilo a la vez pueda acceder a la sección crítica para evitar condiciones de carrera y datos inconsistentes.

Deadlocks

Un deadlock es una situación en la que dos o más hilos se bloquean mutuamente esperando recursos que nunca serán liberados, lo que resulta en un estado de espera infinita.

Tipos de paralelismos

Paralelismo de datos

Se centra en distribuir subconjuntos de los mismos datos en varios núcleos informáticos y realizar la misma operación en cada núcleo.

Paralelismo de tareas

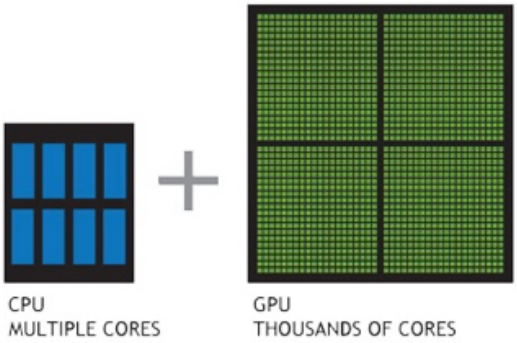
Se enfoca en distribuir tareas a través de mútiples núcleos informáticos. Cada hilo está realizando una operación única. Diferentes hilos pueden estar operando con los mismos datos o pueden estar operando con distintos datos.

CUDA (Compute Unified Device Arquitecture)

Se trata de una plataforma de computación paralela y un modelo de programación creado por NVIDIA. Está diseñado para permitir a los desarrolladores usar las unidades de procesamiento gráfico (GPU) de NVIDIA para realizar cálculos de propósito general más allá de los gráficos tradicionales. CUDA aprovecha la arquitectura de las GPU, que consiste en miles de núcleos de procesamiento que pueden ejecutar miles de hilos (threads) simultáneamente, particularmente útil para tareas que pueden ser paralelizadas, como procesamiento de imágenes, simulaciones científicas y aprendizaje automático.

GPU (Unidad de Procesamiento Gráfico)

La GPU es un procesador formado por muchos núcleos más pequeños y especializados. Al trabajar juntos, los núcleos aumentan el desempeño de forma considerable cuando una tarea de procesamiento puede dividirse entre varios núcleos al mismo tiempo (o en paralelo). La GPU es un componente esencial del gaming moderno, ya que permite obtener imágenes de mejor calidad y una mayor fluidez en el juego. Las GPUs también son útiles en la IA. (Intel, s.f.)



Speedup y eficiencia

Al analizar el programa de forma asintótica, nos damos cuenta de que tiene la misma complejidad. Para determinar la eficiencia de un algoritmo paralelo, se emplea una medida estándar conocida como "**Seed Up**".
El Speed Up de una aplicación que se ejecuta en una máquina paralela de "p" procesadores se define de la siguiente manera:

$$S_p = \frac{T_s}{T_p}$$

Donde:
T_s= Tiempo de ejecución de la implementación secuancial (de un solo hilo)
T_p= Tiempo de ejecución de la implementación paralela con "p" procesadores

La eficiencia es un término relacionado con el Speed Up y se define de la siguiente manera:

$$E = \frac{S_p}{p}$$

Actividad integradora (Suma de números primos)

La actividad integradora consistió en obtener la sumatoria de los números primos dentro de un rango entre 1 y 5000000. Por definición, un número primo es aquel cuya división es entera sólo cuando es dividido entre uno, o entre si mismo. Los números 2, 3, 5, 7, 11, 13, etc. Son algunos ejemplos de números primos, el número 1 no se considera primo.
Para la solución del problema se realizaron tres implementaciones, una scuencial (sin hilos), una paralela (con hilos) y una en CUDA. Para esta última implementación se utilizó la aplicación web **Google Colab**. Esta aplicación proporciona acceso a GPU's de NVIDIA

compatibles con CUDA, lo que permite ejecutar código que aproveche la computación paralela y acelerada por GPU.
Para las tres implementaciones se siguió el mismo algoritmo:

- Si el número (**n**) es menor que 2, es falso.
- Dividir n entre todos los números anteriores (**i**) hasta $\sqrt{n}$. Si n es divisible entre alguno de los números de forma exacta (sin residuo), entonces es falso. Se repite con todos los números siguientes.
- Si el ciclo concluyó de forma normal, entonces se trata de un número primo, verdadero.

Para calcular el tiempo de ejecución de cada una de las implementaciones se utilizó la librería **chrono**. Esta librería nos da acceso a **high_resolution_clock**, el cual será de utilidad debido a su precisión. El tiempo se calcula en milisegundos.

Implementación secuencial

Se creó on arreglo vacío de tamaño 5000000, depués se llenó con números desde el 1 hasta el 5000000, siendo la localidad 4999999, la que contiene el último número. La función **sumarPrimos** recorre todo el arreglo, llamando a la función **esPrimo** para determinar si el número actual es un número primo. En caso de serlo, el número se le suma al acumulador, en caso contrario, ignora el número y el ciclo continúa. Adicionalmente existe la función **contarPrimos** para determinar la cantidad total de números primos (esta función no se toma en cuenta para el tiempo de ejecución). La ejecución se realiza un total de diez veces para así calcular tiempo promedio de ejecución. La implementación secuencial obtuvo el siguiente tiempo:

```
Último elemento = 5000000
Cantidad de números primos = 348513
Suma de los números primos = 838596693108
avg time = 1282.858 ms
```

Implementación paralela

Se creó on arreglo vacío de tamaño 5000000, depués se llenó con números desde el 1 hasta el 5000000, siendo la localidad 4999999, la que contiene el último número.
Para esta implementación es necesario crear los hilos. A cada hilo le corresponde un inicio, un final y un resultado parcial obtenido al sumar, como este programa usa un arreglo, también se requiere de su posición individual. En total se utilizaron 8 hilos para esta implementación.

La función **sumarPrimos** recorre todo el arreglo, llamando a la función **esPrimo** para determinar si el número actual es un número primo. En el ciclo de la suma, cada hilo inicia en un **bloque** y termina en su bloque, esto hace que los hilos no intenten acceder o manipular

los resultados de otros hilos. Cada hilo manipula su propio acumulador y al final deja su resultado, el cual es **parcial**.

En caso de ser un número primo, el número se le suma al acumulador, en caso contrario, ignora el número y el ciclo continúa. **contarPrimos** no se toma en cuenta para el tiempo de ejecución ni para los hilos. La implementación paralela obtuvo el siguiente tiempo:

```
Último elemento = 5000000
Cantidad de números primos = 348513
Suma de los números primos = 838596693108
avg time = 220.028 ms
```

Ahora que obtuvimos el tiempo promedio, podemos calcular el Speed Up:

$$S_p = \frac{1282.858}{220.028} = 5.8304$$

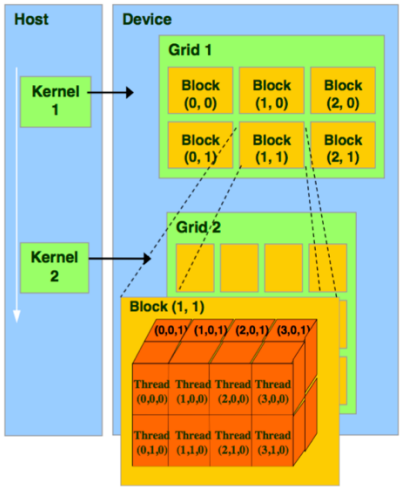
Con el Speed Up, podemos calcular la eficiencia (se utilizaron 8 hilos):

$$E = \frac{5.8304}{8} = 0.7288$$

La eficiencia del programa paralelo con respecto al secuencia fue de un %72.88.

Implementación en CUDA

La implementación en CUDA requirió de cambios notables en el código para adaptar las funciones a la arquitectura. A estas funciones se les llama **kernel**: Nombre de una función ejecutada por CUDA en el GPU. La función paralelizada **sumarPrimos** se convirtió a **global** indicando que es la operación paralelizada. En CUDA, existirá la separación host y device, donde device son operaciones ejecutadas en el GPU y host son las operaciones ejecutadas en el CPU. Para aprovechar las capacidades del GPU, se utilizaron un total de 512 hilos y un total de 32 bloques de hilos (donde cada bloque tiene 512 hilos). La siguiente imagen muestra un ejemplo de bloques:



La función esPrimo se declaró como **device**, una función auxiliar ejecutada en el GPU. Para que la función contarPrimos aun funcionara, fue necesario crear una segunda función para host (**esPrimoHost**). Para calcular el tiempo promedio de forma más "honesta" se ejecutaron las implementaciones secuencial y CUDA en la misma aplicación Google Colab.

```
[ ] !ls
    primos_cuda.cu  sample_data

[1] !g++ primos.cpp -o app

[2] !./app
    Último elemento = 5000000
    Cantidad de números primos = 348513
    Suma de los números primos = 838596693108
    avg time = 4771.469 ms

[ ] !g++ primos_threads.cpp -o app -pthread

[ ] !./app

[3] !nvcc primos_cuda.cu -o app

    !./app
    Último elemento = 5000000
    Cantidad de números primos = 348513
    Suma de los números primos = 838596693108
    avg time = 94.343 ms
```

Con el tiempo promedio hecho con CUDA, podemos calcular el Speed Up:

$$S_p = \frac{4771.469}{94.343} = 50.5758$$

Con el Speed Up, podemos calcular la eficiencia, en este caso se usaron 32 bloques de 512 hilos:

$$E = \frac{50.5758}{512 * 32} = 0.0031$$

La eficiencia del programa paralelo con respecto al secuencia fue de un %0.31.

Reflexiones

El área de la programación paralela avanza cada día conforme los componentes de la computadora progresan. Aactualmente tenemos mejores unidades de memoria, procesadores más pequeños y más eficientes, tarjetas con mayor memoria, tarjetas gráficas con miles de núcleos, etc. Es por esa razón que conocer sobre la programación paralela, en concreto, la basada en hilos, resulta importante y hasta cierto punto, necesario. La gran mayoría de aplicaciones modernas y videojuegos utilizan múltiples hilos, puede ser más demandante y requerir de memoria, pero como a cambio obtenemos mayor velocidad y tiempo de respuesta, y a los usuarios finales les suele interesar más la velocidad que la memoria, al fin y al cabo, las memorias tienen cada vez más espacio.

Realizar las implementaciones secuenciales y paralelas fue una experiencia que me ayudó, primero, a identificar qué es un hilo y donde se aplica, y dos, el tiempo de ahorro que se consigue. Fue interesante hacer una implementación paralela en CUDA, en primer lugar porque no tenía conocimiento claro de una GPU, y porque pude trabajar en una unidad de procesamiento muy poderosa. Es probable que conforme avance en mi carrera profesional como ingeniero en sistemas, aplique hilos en algún momento, y por ello será importante ahondar más en el tema.